Monitoramento e Observabilidade em Redes 5G Baseadas em Software Utilizando eBPF

Ana Beatriz Borges Cordeiro, Bianca Pereira de Carvalho, Jonas Augusto Kunzler, Flávio Geraldo Coelho Rocha

***Resumo*— Este trabalho investiga a aplicação da tecnologia *extended Berkeley Packet Filter* (eBPF) como mecanismo de monitoramento e observabilidade em redes móveis 5G baseadas em *software*. A metodologia baseou-se na estruturação de um ambiente experimental virtualizado e conteinerizado via Docker, utilizando as ferramentas UERANSIM para a emulação do Equipamento do Usuário (*User Equipment*, UE) e da Rede de Acesso Rádio (*Radio Access Network*, RAN), e o Open5GS para as funções de núcleo da rede 5G (*5G Core*, 5GC). Analisou-se o desempenho e a eficiência dos ganchos *eXpress Data Path* (XDP) e *Traffic Control* (TC) integrados à Função de Plano de Usuário (*User Plane Function*, UPF), estabelecendo um comparativo com ferramentas tradicionais no espaço do usuário, como o `tcpdump`, com foco no consumo de CPU. Como evolução da investigação, foi avaliada a implementação de um sistema de aprendizado de máquina diretamente no plano de dados (*In-Kernel ML*), que utiliza um modelo de *Random Forest* para realizar a classificação de tráfego em tempo real entre fluxos de Banda Larga Móvel aprimorada (*enhanced Mobile Broadband*, eMBB) e de Comunicações Ultraconfiáveis e de Baixa Latência (*Ultra Reliable Low Latency Communications*, URLLC). Os resultados indicaram que a coleta de métricas via eBPF apresentou menor *overhead* computacional, com médias de uso total de CPU de 14,51% (XDP) e 13,99% (TC), em relação aos 16,93% do `tcpdump`. Adicionalmente, a classificação de tráfego no *kernel* demonstrou alta viabilidade, atingindo acurácia de 99,98% e tempo de inferência de cerca de 0,22 μs. A análise conclui que a abordagem apresenta aderência aos requisitos de eficiência e baixa latência do 5G, fornecendo uma base para a continuidade da pesquisa.**

***Palavras-Chave*— eBPF, ganchos, kernel, 5G**

***Abstract*— This work investigates the application of extended Berkeley Packet Filter (eBPF) technology as a monitoring and observability mechanism in software-based 5G mobile networks. The methodology was grounded in setting up a virtualized and containerized experimental environment using Docker, utilizing the UERANSIM tool for User Equipment (UE) and Radio Access Network (RAN) emulation and Open5GS for the 5G Core (5GC) network functions. Initially, the performance and efficiency of eXpress Data Path (XDP) and Traffic Control (TC) hooks integrated into the User Plane Function (UPF) were analyzed, establishing a technical comparison with traditional user-space tools, such as `tcpdump`, with focus on CPU consumption. As a further development of the investigation, the implementation of a machine learning system directly in the data plane (In-Kernel ML) was evaluated, using a Random Forest model to perform real-time traffic classification between enhanced Mobile Broadband (eMBB) and Ultra Reliable Low Latency Communications (URLLC) flows. The results indicated that metric collection via eBPF presented lower computational overhead, with average total CPU usage of 14.51% (XDP) and 13.99% (TC), compared to 16.93% for tcpdump. In addition, in-kernel traffic classification demonstrated high feasibility, achieving an accuracy of 99.98% and an inference time of approximately 0.22 μs. The analysis concludes that the approach presents adherence to the efficiency and low-latency requirements of 5G, providing a foundation for further research.**

***Keywords*— eBPF, hooks, kernel, 5G**

I. Introdução

As redes móveis de quinta geração (5G) foram desenvolvidas para atender a um conjunto diverso de aplicações, abrangendo desde serviços de Banda Larga Móvel aprimorada (*enhanced Mobile Broadband*, eMBB), caracterizados pelo alto volume de dados, até Comunicações Ultraconfiáveis e de Baixa Latência (*Ultra Reliable and Low Latency Communications*, URLLC), exigidas por aplicações críticas [1]. Para suportar essa diversidade de requisitos, a arquitetura 5G adota um modelo baseado em serviços (*Service-Based Architecture*, SBA) e promove a separação entre os planos de controle e de usuário (*Control and User Plane Separation*, CUPS) [2], estruturando o sistema em três componentes: o Equipamento do Usuário (*User Equipment*, UE), a Rede de Acesso por Rádio (*Radio Access Network*, RAN) e o núcleo da rede (*5G Core*, 5GC) [3].

No plano de usuário, a Função de Plano de Usuário (*User Plane Function*, UPF) concentra o encaminhamento, a inspeção de pacotes e a aplicação de regras de qualidade de serviço (*Quality of Service*, QoS). Por ser o nó pelo qual todo o tráfego das aplicações trafega, o processamento tradicional de pacotes nessa função pode se tornar um gargalo em cenários de tráfego intenso [2]. Esse aspecto torna o monitoramento do plano de dados essencial e desafiador, uma vez que ferramentas tradicionais de captura operam no espaço do usuário do sistema, como o `tcpdump`, e introduzem sobrecarga adicional por exigirem cópias de dados e trocas de contexto entre o *kernel* e o espaço do usuário [2].

Nesse contexto, o *extended Berkeley Packet Filter* (eBPF) se destaca como uma alternativa às ferramentas tradicionais. O eBPF consiste em uma infraestrutura integrada ao *kernel* Linux que permite executar programas em ambientes isolados (*sandboxes*) diretamente no núcleo do sistema operacional, sem a necessidade de modificar seu código-fonte ou carregar módulos dinâmicos do *kernel* [4]. Por meio de pontos de ancoragem (*hooks*) na pilha de rede como o *eXpress Data Path* (XDP) e o *Traffic Control* (TC), o eBPF viabiliza a coleta de métricas e o processamento de pacotes com baixa latência e reduz o consumo de recursos [5].

Este trabalho investiga a aplicação do eBPF como mecanismo de monitoramento e observabilidade em redes 5G baseadas em *software*. Para isso, foi estruturado um ambiente experimental virtualizado e conteinerizado via Docker, empregando o UERANSIM para a emulação de rádio e o Open5GS para as funções de núcleo da rede. A investigação foi conduzida em duas frentes complementares. Na primeira, avaliou-se o custo computacional da inspeção passiva de pacotes nos

hooks XDP e TC integrados à UPF, em comparação com a ferramenta tradicional `tcpdump`, sob métricas de uso de CPU. Na segunda, como evolução da investigação, avaliou-se um uso ativo do eBPF: a reprodução de um sistema de aprendizado de máquina diretamente no plano de dados (*In-Kernel ML*), baseado em um modelo *Random Forest* para classificação de tráfego em tempo real entre fluxos eMBB e URLLC, desenvolvido no trabalho de Galvão et al. [2].

O restante deste trabalho está organizado da seguinte maneira. A Seção II apresenta a fundamentação teórica sobre a arquitetura de redes 5G e o eBPF. A Seção III descreve a configuração experimental adotada. A Seção IV detalha a metodologia de avaliação. A Seção V apresenta a aplicação do caso de uso de classificação de tráfego. A Seção VI discute os resultados obtidos e, por fim, a Seção VII apresenta as conclusões do trabalho.

II. Fundamentação Teórica

Esta seção aborda os conceitos essenciais que embasam este trabalho, são eles: arquitetura de redes 5G e *extended Berkeley Packet Filter* (eBPF).

A. Arquitetura de Redes 5G e o Plano de Usuário

A rede 5G adota uma arquitetura baseada em serviços, SBA, e implementa a separação entre os planos de controle e de usuário, CUPS [2]. Essa arquitetura foi projetada para ter uma implementação flexível e atender demandas e requisitos rigorosos, como alta confiabilidade e latência ultra baixa exigidas por aplicações como cirurgias remotas [2].

A estrutura geral do sistema 5G é mostrada na Fig. 1. Ela é composta pelo equipamento do usuário, pela rede de acesso por rádio, e pelo núcleo da rede, na Fig. 1 designados como UE, NG-RAN e 5GC, respectivamente [3].



Fig. 1
Visão geral da rede 5G

Fonte: 3GPP [3]

O núcleo do 5G é caracterizado pela modularidade e independência das suas funções de rede (*Network Functions*, NFs), além de ter uma separação clara entre os planos de controle e de usuário, como mostra a Fig. 2 [3]. No plano de controle destacam-se a Função de Gerenciamento de Acesso e Mobilidade (*Access and Mobility management Function*, AMF) e a Função de Gerenciamento de Sessão (*Session Management Function*, SMF); e no plano de usuário destaca-se a Função de Plano de Usuário (*User Plane Function*, UPF).

O fluxo de comunicação se inicia com uma requisição do UE para estabelecer uma conexão para a transmissão de dados,

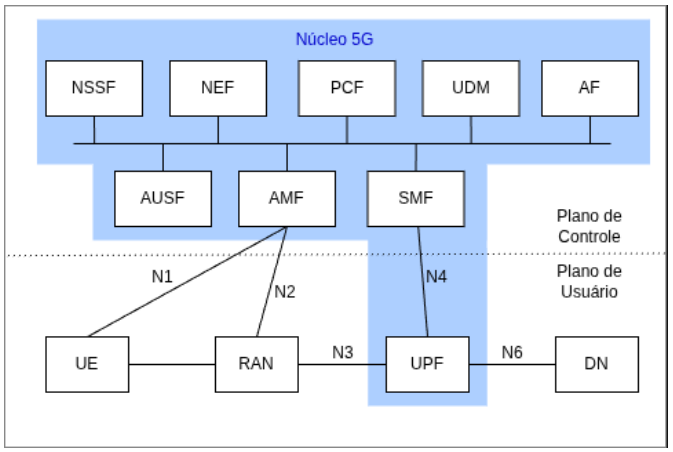


Fig. 2
Arquitetura da rede 5G

Fonte: Adaptado de [3]

a AMF recebe essa requisição e faz o registro e autenticação do dispositivo [6]. A SMF lida com as funções gerais de gerenciamento de sessão e com a alocação de endereço IP para o UE [7]. A RAN é responsável pelo roteamento dos dados do plano de usuário para a UPF, que atua como o nó central no processamento do tráfego e é o ponto de conexão entre a estação rádio-base do 5G (*gNodeB*, gNB) e a rede de dados externa (*Data Network*, DN) [7]. Após essa etapa, o tráfego de dados flui pelo plano de usuário, onde a UPF é responsável pelo encaminhamento dos pacotes, pela classificação dos fluxos de tráfego e pela aplicação das regras de QoS definidas pelo núcleo da rede [6].

No sistema 5G, dois dos principais serviços são o URLLC e o eMBB [1]. O URLLC requer ultraconfiabilidade e baixa latência, enquanto o eMBB é caracterizado pelo grande volume de dados transmitidos [3]. Tais serviços exigem um alto desempenho da rede, porém, como todo o tráfego gerado pelas aplicações passa pela UPF, o processamento tradicional de pacotes dessa função pode se tornar um gargalo em cenários de tráfego intenso [2].

Para contornar esse gargalo de processamento e garantir o cumprimento dos requisitos do 5G, novas abordagens vêm sendo desenvolvidas [2]. Nesse cenário, destaca-se o eBPF, uma tecnologia que tem ganhado relevância por permitir otimizações de rede dinâmicas e eficientes diretamente no *kernel* do sistema operacional. O uso do eBPF surge, portanto, como uma alternativa promissora para melhorar a eficiência das UPFs.

B. Extended Berkeley Packet Filter (eBPF)

O eBPF consiste em uma infraestrutura tecnológica integrada ao *kernel* Linux que viabiliza a execução de programas em ambientes isolados (*sandboxes*) dentro do núcleo do sistema operacional. Conforme abordado pela documentação oficial da comunidade eBPF.io [4], essa arquitetura permite estender as capacidades funcionais do *kernel* de maneira segura e eficiente. Consequentemente, elimina-se a necessidade de

modificação direta do código-fonte do sistema operacional ou do carregamento de módulos dinâmicos de núcleo (*Loadable Kernel Modules*), procedimentos que historicamente poderiam comprometer a estabilidade do sistema.

A tecnologia caracteriza-se como uma evolução do *Berkeley Packet Filter* (BPF) original, concebido para otimizar a filtragem de pacotes em ferramentas de monitoramento de rede operando na forma de uma máquina virtual minimalista interna ao *kernel*.

1) ***Ciclo de vida do eBPF****:* O ciclo de vida de um programa eBPF é composto por fases estritamente definidas que garantem segurança e desempenho ao estender o *kernel* Linux sem a necessidade de reinicialização, como mostrado na Fig. 3.

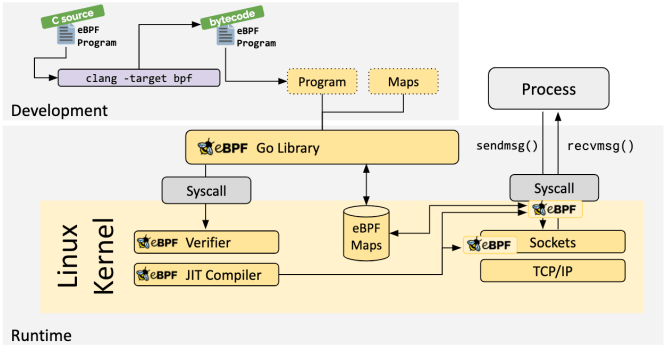


Fig. 3

ARQUITETURA DE CARREGAMENTO E VERIFICAÇÃO DO EBPF

Fonte: eBPF.io [4]

O processo se inicia no espaço do usuário (*userspace*), onde o desenvolvedor escreve a lógica do programa em linguagens de alto nível. Através de uma suíte de compiladores como o LLVM/Clang, esse código é transformado em um arquivo que contém o *bytecode* eBPF genérico e independente de arquitetura, pronto para ser processado pelo núcleo do sistema.

Antes de qualquer instrução ser executada, o código deve passar obrigatoriamente pelo Verificador (*Verifier*), que analisa todos os caminhos possíveis de execução para garantir que o programa seja seguro: ele valida se o processo possui privilégios suficientes (como `CAP_BPF` ou *root*), assegura que não exista acessos a memória fora dos limites ou variáveis não inicializadas e garante que o programa sempre termine, proibindo loops infinitos que poderiam levar ao travamento do *kernel* e definindo um limite de instruções por programa. Além disso, o ambiente de execução do eBPF impõe que os programas não utilizem aritmética de ponto flutuante, de modo que os cálculos no *kernel* sejam realizados com aritmética inteira.

A próxima fase é o acoplamento (*attachment*) do programa a um ponto de ancoragem (*hook*) predefinido no sistema [4]. Dependendo da finalidade da aplicação, o programa pode ser vinculado a ganchos do subsistema de rede, como o XDP ou o TC, ou a pontos de instrumentação do sistema, como ganchos de rastreamento dinâmicos (*kprobes*, *uprobes*) e estáticos (*tracepoints*).

Com o acoplamento concluído, o programa assume o estado operacional ativo. A execução ocorre de maneira estritamente orientada a eventos onde a cada instância em que o evento associado ao *hook* é disparado pelo sistema operacional, o núcleo invoca o código eBPF compilado pelo compilador *Just-In-Time* (JIT) [4], [5].

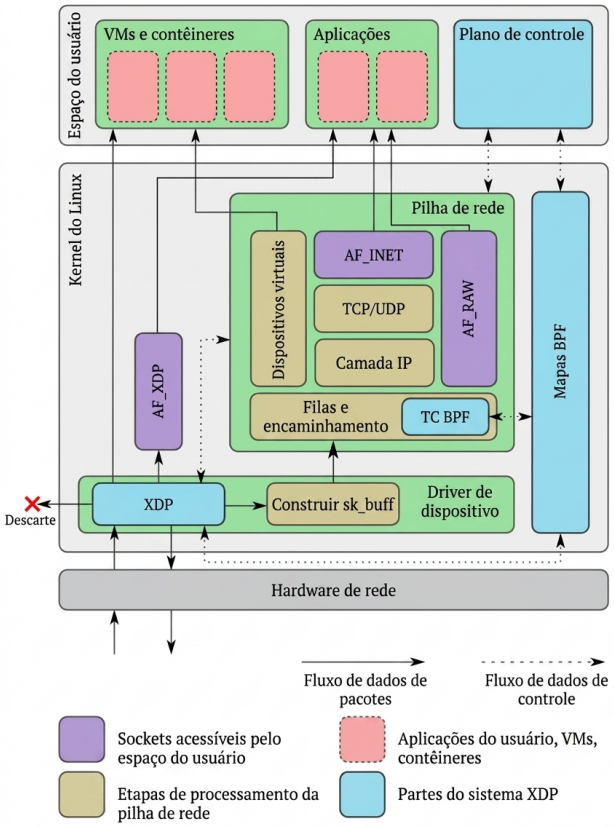


Fig. 4

INTEGRAÇÃO DE PROGRAMAS EBPF (*hooks* XDP E TC) NA PILHA DE REDE DO *kernel* LINUX E COMUNICAÇÃO COM O ESPAÇO DO USUÁRIO.

Fonte: Adaptado de [5]

2) ***Hooks XDP e TC****:* Como ilustrado na Fig. 4, o *hook* XDP se localiza no nível mais baixo possível da pilha de rede do Linux, permitindo que o programa eBPF intercepte e atue sobre os dados assim que chegam no *hardware*, antes mesmo do *kernel* alocar estruturas de dados complexas como o `sk_buff` ou realizar qualquer processamento pesado. Contudo, segundo a documentação oficial da comunidade eBPF.io [4], o XDP pode operar em três modos distintos condicionados ao suporte oferecido pelo *driver* de rede da interface. São eles:

- **XDP Nativo (DRV_MODE, *Driver Mode*)**: o programa é executado no nível mais baixo da pilha de rede a nível de *software*, antes da alocação do `sk_buff`.
- **XDP *Offloaded* (HW_MODE, *Hardware Mode*)**: o programa é executado diretamente no *hardware* da placa de rede (NIC), liberando a CPU do *host* do processamento dos pacotes. Esse modo depende do suporte específico das placas de rede a essa funcionalidade.
- **XDP Genérico (SKB_MODE, *Socket Buffer Mode*)**: é o modo com menos benefícios e sem dependência do *driver* de rede. O programa é acionado após a

alocação do `sk_buff`, em um ponto mais avançado da pilha de rede. Por essa razão, parte das vantagens de desempenho associadas ao processamento antecipado do XDP é reduzida, aproximando seu comportamento ao do *hook* TC.

Quando o *driver* não oferece suporte ao modo nativo, o *kernel* realiza o acoplamento automático para o modo genérico, garantindo a execução do programa.

Ao finalizar a execução, o programa XDP emite um código de retorno que define o destino do pacote, como ilustrado na Fig.5. Os códigos de retorno são:

- **XDP_PASS**: encaminha o pacote para a pilha de rede convencional do *kernel* para processamento normal.
- **XDP_DROP**: descarta o pacote imediatamente no nível do *driver*.
- **XDP_TX**: retransmite o pacote de volta pela mesma interface de rede em que foi recebido.
- **XDP_REDIRECT**: permite desviar o tráfego para outra interface de rede, para uma CPU diferente ou diretamente para uma aplicação no espaço do usuário via *sockets* AF_XDP.

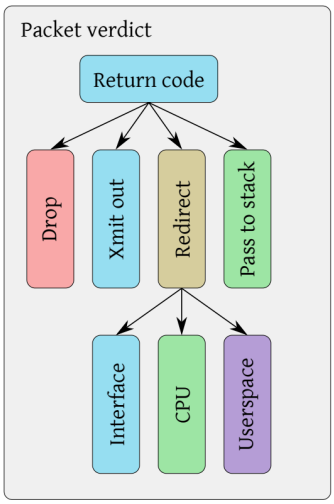


Fig. 5
CÓDIGOS DE RETORNO XDP

Fonte: [5].

A arquitetura do XDP possibilita que funções de rede críticas sejam realizadas com latência mínima e baixo consumo de CPU ao evitar cópias de dados e trocas de contexto desnecessárias.

Por outro lado, o *hook* TC se posiciona em uma camada subsequente da pilha de rede do Linux comparado ao XDP [8]. Diferentemente do XDP, o TC executa suas instruções após a alocação da estrutura de dados `sk_buff` pelo núcleo [9]. Essa característica permite que o TC intercepte e manipule pacotes tanto no fluxo de entrada (*ingress*) quanto no de saída (*egress*) das interfaces, atuando diretamente sobre o tráfego que ingressa (*Traffic In*) e que deixa (*Traffic Out*) o plano de dados da função de rede [10].

*3) **Mapas eBPF**:* Como mencionado anteriormente, os programas eBPF são disparados em resposta a eventos específicos no *kernel*. Devido à sua arquitetura de execução isolada (*sandbox*), o programa retorna ao seu estado inicial a cada nova ativação, não possuindo acesso a memória persistente dentro de seu próprio contexto de execução. Para viabilizar a persistência de dados, os mapas eBPF funcionam como um meio de armazenamento compartilhado entre sucessivas execuções, permitindo também a coordenação entre diferentes programas eBPF no *kernel* e a comunicação eficiente com o espaço do usuário.

Os mapas eBPF podem ser classificados em alguns tipos [4], tais quais:

- **Tipos Genéricos**: Mapas mais versáteis e amplamente utilizados para armazenamento geral de dados, como o `BPF_MAP_TYPE_HASH` e o `BPF_MAP_TYPE_ARRAY`
- **Mapas por CPU (Per-CPU)**: Mapas que mantêm uma cópia separada para cada núcleo lógico da CPU, como os mapas `BPF_MAP_TYPE_PERCPU_HASH` e `BPF_MAP_TYPE_PERCPU_ARRAY`
- **Streaming de Dados**: Mapas projetados para transferir grandes volumes de informações do *kernel* para o espaço do usuário de forma eficiente. Exemplos: `BPF_MAP_TYPE_PERF_EVENT_ARRAY` e `BPF_MAP_TYPE_RINGBUF`
- **Mapas de Mapas**: Mapas que permitem que o valor armazenado seja referência a outro mapa eBPF, como o `BPF_MAP_TYPE_ARRAY_OF_MAPS`

III. CONFIGURAÇÃO EXPERIMENTAL

O sistema 5G foi simulado em um ambiente conteinerizado utilizando Docker em uma Máquina Virtual (*Virtual Machine*, VM). As especificações técnicas da VM estão resumidas na Tabela I, e a Tabela II mostra a estrutura de contêineres dos principais componentes da rede 5G.

TABELA I
ESPECIFICAÇÕES TÉCNICAS DA VM

Sistema Operacional	Ubuntu 22.04.5 LTS
Kernel	5.15.0-176-generic
Memória	32 GiB
Quantidade de núcleos de CPU	8
Virtualização	VT-x
Hipervisor	KVM

TABELA II
RELAÇÃO RESUMIDA DE CONTÊINERES DO AMBIENTE 5G SIMULADO

Contêiner	**Componente**	**Interface**	**IP**
b-ue	UE	net-n2	10.120.0.200
-	UE	ue-subnet	10.160.0.10
b-gnb	gNB	net-n2	10.120.0.100
-	gNB	net-n3	10.130.0.100
b-upf	UPF	net-n3	10.130.0.21
-	UPF	net-n4	10.140.0.21
-	UPF	net-n6	10.150.0.21
b-dn	DN	net-n6	10.150.0.100

A. Topologia da Rede

A topologia de rede foi segmentada em cinco interfaces virtuais do tipo *bridge*, mapeando as interfaces de referência da arquitetura 5G, como detalha a Tabela III.

TABELA III
INTERFACES VIRTUAIS (*bridges*)

Interface	IP Subnet	Objetivo
SBI	10.110.0.0/16	Comunicação entre as NFs de controle
N2	10.120.0.0/16	Sinalização entre o UE e a AMF
N3	10.130.0.0/16	Tráfego via túnel GTP-U [1] entre o gNB e a UPF
N4	10.140.0.0/16	Comunicação entre a UPF e a SMF
N6	10.150.0.0/16	Tráfego entre a UPF e a rede externa

Para a implementação do núcleo da rede, foi adotado o *software* de código aberto Open5GS na versão 2.7.6. O plano de controle do 5GC é composto pelas funções de rede AMF, SMF, NRF, SCP, UDR, UDM, AUSF, PCF e NSSF[2], com os dados e perfis de assinantes sendo armazenados de forma persistente em um banco de dados MongoDB versão 7.0.

No plano de usuário, para viabilizar a injeção e execução de códigos eBPF na interface de rede, o contêiner da UPF opera em modo privilegiado com capacidades elevadas (NET_ADMIN e SYS_ADMIN), mapeando os volumes de módulos do *kernel* hospedeiro (/lib/modules e /sys/kernel/btf) bem como o sistema de montagem local do BPF (/sys/fs/bpf).

O *software* UERANSIM foi utilizado para implementar o UE e a RAN. E, finalmente, a DN é um contêiner que conta com softwares como o `iperf3` com a finalidade de atuar como destino para o tráfego gerado no UE no sentido *uplink*.

B. Fluxo de Comunicação no Plano de Usuário

O tráfego de dados inspecionado no experimento é gerado no UE com destino à DN, como mostra a Fig. 6. Os pacotes de dados saem pela interface `uesimtun0` do UE e são encaminhados para a DN através da gNB e da UPF que se comunicam por meio da interface N3, onde foram posicionadas as ferramentas de monitoramento de pacotes na UPF.

C. Características do Docker

O Docker implementa redes do tipo *bridge*, utilizadas no experimento deste trabalho, por meio de pares de interfaces Ethernet virtuais (*veth pairs*). Por padrão, cada contêiner recebe uma interface virtual que o conecta à *bridge*. Como mostra a Fig. 7, uma das interfaces do par é exposta no *host*, fora do contêiner e ligada à *bridge*, e a outra dentro do contêiner [11] [12], ou seja, o tráfego quando flui pela interface N3, configurada como *bridge*, por exemplo, passa por quatro interfaces virtuais do

[1]Protocolo de Tunelamento GPRS para o Plano do Usuário, do inglês *GPRS Tunnelling Protocol User Plane*

[2]NRF: *Network Repository Function*; SCP: *Service Communication Proxy*; UDR: *Unified Data Repository*; UDM: *Unified Data Management*; AUSF: *Authentication Server Function*; PCF: *Policy Control Function*; NSSF: *Network Slice Selection Function*.

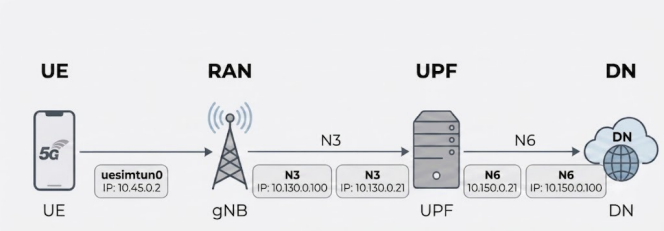


Fig. 6
FLUXO DO TRÁFEGO DE DADOS NA REDE SIMULADA

Fonte: Produção própria.

Docker, que são os dois pares *veth* da *bridge* correspondente à interface N3: um par do contêiner da UPF e outro da gNB. Esse detalhe de implementação é relevante para a metodologia adotada, pois a existência de pares *veth* no *host* fundamenta tanto a medição de CPU no nível do *host* (Seção IV-C) quanto a escolha da interface de captura no estudo de caso (Seção V-A).

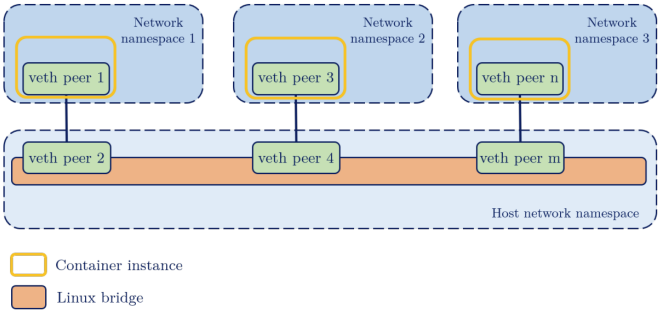


Fig. 7
ESTRUTURA DAS *veth pairs* E *bridge* NO AMBIENTE DOCKER

Fonte: Özturk, T. (2021) [13]

A presença dessas interfaces virtuais e de uma ponte virtual para a comunicação entre os contêineres faz com que o pacote percorra um caminho mais longo na pilha de rede do que ocorre na comunicação entre processos nativos. No modo *bridge* cada contêiner possui um espaço de nomes (*namespace*) de rede isolado e todo o tráfego precisa atravessar a ponte e etapas adicionais de processamento de pacotes, o que prolonga o caminho e aciona rotinas extras no *kernel* [11] [12]. Como consequência, essas etapas adicionais estão associadas a maior consumo de CPU além de redução de vazão e aumento de latência em relação à comunicação direta [11] [12].

D. Programas eBPF

Os programas eBPF anexados aos *hooks* XDP, no modo genérico, e TC, no fluxo de entrada, foram desenvolvidos com o objetivo de avaliar o custo computacional da inspeção de pacotes. Optou-se pelo modo genérico porque, embora as interfaces *veth* suportem o modo nativo desde a versão 4.19 do *kernel* [13], sua habilitação no ambiente Docker requer que outro programa XDP seja anexado no par externo da *veth* [13] [14], ficando sua avaliação como trabalho futuro.

Para viabilizar a comparação de consumo de CPU entre essas tecnologias (que rodam no *kernel*) e o `tcpdump` (que opera no espaço do usuário), os códigos realizam a análise de cabeçalhos, o cálculo de *throughput* e do tempo entre chegadas dos pacotes (*Inter-Arrival Time*, IAT). Nesse contexto, a extração da métrica de rede funciona como uma carga de trabalho (*workload*) para forçar o processamento e gerar o estresse necessário para as medições.

Os programas foram desenvolvidos em linguagem C utilizando a biblioteca *libbpf*, que é a ferramenta de referência integrada ao *kernel* Linux para a construção de aplicações eBPF [15]. As ferramentas eBPF desenvolvidas neste trabalho são compostas por duas partes: um código executado no *kernel* para inspecionar os pacotes, extrair as métricas e armazená-las em um mapa eBPF, e um código no espaço de usuário que lê essas informações do mapa e as exibe no terminal em tempo real. Ambos os programas (XDP e TC) foram anexados na interface N3 da UPF.

Os cálculos da latência e da perda de pacotes não foram incluídos por se tratarem de métricas de rede que para serem mensuradas precisam de uma visualização completa do tráfego. A perda de pacotes é medida comparando a quantidade de requisições enviadas e a quantidade de respostas recebidas, e a diferença entre as duas resulta na quantidade de pacotes perdidos. Da mesma forma, para calcular a latência é necessário comparar o momento que a requisição foi enviada e o momento em que a resposta foi recebida. Ambas as métricas precisam que a ferramenta de medição tenha um acesso bidirecional ao tráfego. Como os *hooks* utilizados atuam em direções específicas do fluxo e a implementação utilizada acopla o programa em apenas uma direção da interface (N3, de *uplink*), não há a visibilidade bidirecional simultânea necessária para correlacionar esses eventos.

IV. Metodologia

Com base no cenário de simulação da rede 5G detalhado, esta seção apresenta os métodos e procedimentos utilizados para avaliar o custo computacional do eBPF comparado ao utilitário `tcpdump`. As subseções a seguir detalham a lógica dos programas desenvolvidos para os *hooks* XDP e TC e o funcionamento do `tcpdump`, e, por fim, são descritos os cenários analisados e a maneira como os experimentos foram conduzidos.

A. Ferramentas de Monitoramento

1) Programa eBPF/XDP: O XDP recebe o contexto `struct xdp_md`, que fornece acesso direto aos dados do pacote por meio dos ponteiros `data` e `data_end`, os quais indicam, respectivamente, o início e o fim da área de memória que contém o pacote. O programa é executado na UPF, onde ele desencapsula o pacote, que chega encapsulado em GTP-U, de forma a obter o pacote original enviado pelo UE. Após isso, o programa extrai do cabeçalho IP os endereços de origem e destino e o tamanho total do pacote, e calcula o IAT a partir do *timestamp* do último pacote observado para o mesmo IP de origem, mantido em um mapa eBPF. Essas informações são encapsuladas em uma estrutura de evento que é enviada do *kernel* para o espaço do usuário de forma assíncrona usando um *Ring Buffer* (`BPF_MAP_TYPE_RINGBUFF`). No programa em C do espaço do usuário o *throughput* é de fato calculado. O código mantém uma estrutura de estado onde acumula os bytes recebidos. O cálculo do *throughput* em Mbps ocorre no espaço do usuário a cada intervalo de consolidação, definido como uma janela de temporal periódica de 1 segundo.

2) Programa eBPF/TC: O gancho TC atua após a alocação de memória pelo sistema operacional, recebendo a estrutura `struct __sk_buff`. A extração de cabeçalhos (Ethernet, IP, UDP, GTP) é feita utilizando a função auxiliar `bpf_skb_load_bytes()`. Após isso, o programa realiza o mesmo procedimento do programa eBPF/XDP.

3) Utilitário `tcpdump`: Diferente das implementações baseadas em eBPF que operam diretamente no *kernel*, o utilitário `tcpdump` funciona no espaço do usuário, utilizando a biblioteca *libpcap* para a captura de dados. O funcionamento exige que o pacote percorra um caminho mais longo na pilha de rede do Linux antes de ser processado.

Para que o cálculo do *throughput* ocorra, os pacotes precisam ser copiados da memória do *kernel* para o espaço do usuário, um processo que envolve operações de cópia de dados e trocas de contexto (*context switching*) sucessivas entre o *kernel* e a aplicação.

B. Cenários de Avaliação

Para comparar o impacto das ferramentas de coleta de métricas de rede (XDP, TC e `tcpdump`), avaliou-se o uso de CPU do ambiente submetido a uma carga de tráfego constante na UPF. A análise foi dividida nos seguintes cenários:

1) *Baseline* (Linha de base): é o cenário de referência, onde flui o mesmo tráfego dos demais cenários, mas não há execução de programas de extração de métricas de desempenho de rede;
2) XDP: extração de métricas no espaço de *kernel* por meio de um programa eBPF anexado ao *hook* XDP no modo genérico;
3) TC: extração de métricas no espaço de *kernel* por meio de um programa eBPF anexado ao *hook TC Ingress*;
4) `tcpdump`: extração de métricas no espaço de usuário utilizando a ferramenta nativa `tcpdump`.

C. Execução

A metodologia para a execução do experimento consistiu na geração de tráfego, na execução das ferramentas XDP, TC e `tcpdump`, e no monitoramento de CPU automatizados por meio de *scripts bash* desenvolvidos especificamente para executar o experimento em cada cenário. Além disso, para assegurar o isolamento das métricas e evitar que o estresse computacional de um teste influenciasse o seguinte, o ambiente Docker foi reinicializado entre as execuções de cada cenário.

O procedimento seguiu as seguintes etapas:

1) Etapa 1 - Reinicialização e Estabilização do Ambiente: Antes de avaliar cada cenário, o serviço do Docker e os contêineres que compõem a rede 5G foram reiniciados para evitar interferências de testes anteriores. A execução do *script* de cada cenário ocorreu apenas após a estabilização do sistema

e a confirmação de que a interface de túnel (`uesimtun0`) do UE foi estabelecida. Essa validação foi feita verificando os *logs* do contêiner do UE:

```
bianca@ebpf:~/ambiente_5g$ docker logs b-ue
UERANSIM v3.2.8
[...]
[2026-05-26 00:00:00.800] [app] [info]
Connection setup for PDU session[1] is
successful, TUN interface[uesimtun0, 10.45.0.2]
is up
```

2) Etapa 2 - Compilação dos Programas eBPF: Nos cenários correspondentes ao XDP e TC, antes da execução do *script bash*, os programas eBPF desenvolvidos foram compilados dentro do contêiner da UPF, deixando os binários de execução prontos para serem iniciados pelo *script* na Etapa 3.

3) Etapa 3 - Configuração dos Serviços e Ferramentas: Com o ambiente reinicializado e estabilizado e com o binário do programa eBPF gerado, o *script* de automação foi acionado para realizar a geração de tráfego, extração de métricas de rede e de uso de CPU. O experimento foi repetido 30 vezes.

A fim de viabilizar o recebimento do tráfego oriundo da UE, em todas as iterações foi iniciado o servidor `iperf3` no contêiner da DN com o seguinte comando:

```
docker exec -d b-dn iperf3 -s
```

Em seguida, a ferramenta de monitoramento correspondente ao cenário foi iniciada na interface N3 da UPF. Para o *baseline*, nenhum programa foi anexado nem o `tcpdump` executado. Para o XDP e TC, os programas eBPF foram anexados na interface N3 da UPF através da execução do respectivo binário no contêiner da UPF. Para o `tcpdump`, a captura foi feita filtrando o tráfego de encapsulamento GTP-U na porta 2152:

```
tcpdump -i eth1 -n -nn -w /dev/null udp port
2152
```

Onde a interface `eth1` especificada no comando corresponde à interface N3 da UPF, enquanto a porta UDP 2152 restringe a captura ao tráfego de encapsulamento GTP-U.

Com a ferramenta de coleta de métricas em execução e com o servidor `iperf3` ativo, foi gerado um tráfego UDP simulado a partir do UE, parametrizado com pacotes de 700 bytes e taxa de transmissão de 100 Mbps. Esse tamanho foi adotado visando evitar problemas de fragmentação decorrentes do encapsulamento GTP-U e maximizar o *packet rate*, intensificando a carga sobre o *kernel*

```
docker exec -d b-ue iperf3 -c 10.150.0.100 -u -b
 100M -l 700 -t 35
```

Em seguida, foi iniciada a medição de CPU no *host* (VM), pois os contêineres Docker não possuem *kernel* próprio e os programas eBPF são carregados e executados no *kernel* do *host*, ou seja, o custo de processamento dos programas eBPF ocorrem no espaço do *kernel* do *host*. O monitoramento foi feito utilizando o utilitário `mpstat`, parametrizado para registrar o uso de CPU a cada 1 segundo e gravar o resultado em um arquivo de texto:

```
mpstat -P ALL 1 20 > mpstat_cenario_xdp_1.txt
```

Uma vez que a captura do uso de CPU é realizada no *host*, os processos da UPF precisam ser atribuídos a um núcleo de CPU específico. Para isso, por meio do Docker (`cpuset`), foram definidas as CPUs virtuais (vCPUs) dos contêineres, como mostra a Tabela IV. Além disso, para direcionar o processamento dos pacotes que fluem pelas interfaces da UPF às mesmas vCPUs atribuídas ao contêiner, o *Receive Packet Steering* (RPS) [3] foi configurado de modo que os pacotes que fluem pelas interfaces da UPF sejam processados nas mesmas vCPUs atribuídas ao contêiner da UPF.

TABELA IV
CONTÊINERES E VCPUS

Contêineres/Componentes	**vCPUs**
Banco de dados e NFs de controle	0 e 1
UPF	2 e 3
gNB	4 e 5
UE e DN	6 e 7

Ao final da coleta do `mpstat` em cada iteração o *script* encerra os processos do `iperf3`, correspondentes a geração e recebimento de tráfego, o processo da ferramenta de extração de métricas de rede e o `mpstat`. E antes de iniciar uma nova iteração o *script* aguarda 40 segundos para que a rede e a CPU retornem ao estado ocioso. As saídas de cada uma das 30 iterações são automaticamente exportadas para arquivos de texto, servindo como base padronizada para a extração dos dados e geração dos gráficos de visualização na seção de resultados.

V. APLICAÇÃO DE CASO DE USO

As seções anteriores avaliaram o eBPF como ferramenta de monitoramento passivo, medindo o *overhead* de inspeção de pacotes nos *hooks* XDP e TC em comparação ao `tcpdump`. Esta seção aborda um caso de uso onde o eBPF é utilizado de forma ativa para realizar a classificação de tráfego de rede em tempo real, com o objetivo de reproduzir o trabalho desenvolvido por Galvão et al. [2] no ambiente experimental descrito na Seção III.

O projeto foi configurado em um ambiente experimental composto por três Máquinas Virtuais (VMs) rodando Ubuntu 20.04 que emulam uma rede 5G utilizando o UERANSIM, para simular o UE e a gNB, e o Free5GC, para simular o núcleo da rede. O programa eBPF é anexado no *hook* XDP da interface de rede da UPF. O gerenciamento do eBPF é feito em Python utilizando o *framework* BCC para o carregamento do *bytecode*, enquanto a lógica de decisão do modelo *Random Forest* é convertida para C compatível com o *kernel* através da biblioteca *emlearn*. Para superar as restrições do verificador eBPF quanto ao número de instruções, utiliza-se *tail calls* (chamadas de cauda). Tail calls são um recurso nativo do eBPF que permite o encadeamento de múltiplos programas, possibilitando que a execução seja transferida de um programa para outro [2]. No projeto, elas foram utilizadas dividindo o classificador em 3 programas encadeados:

- *Model Part1*: responsável por avaliar as árvores de decisão de 0 a 3 e atualizar a contagem global de votos.

[3]Mecanismo de *software* do Linux que distribui o processamento de pacotes recebidos entre CPUs específicas [16].

- *Model Part2*: disparado via *tail call* pelo programa anterior para processar as árvores de 4 a 6.
- *Model Part3*: encarregado de avaliar as árvores restantes (7 a 9) e realizar a votação final por maioria para definir a classe do tráfego.

Além disso, para a execução do classificador no espaço do *kernel* foi utilizada aritmética inteira, devido a restrição do eBPF ao uso de ponto flutuante.

O classificador avalia dois tipos de tráfego: URLLC e eMBB. O URLLC é rotulado como classe 0 e é caracterizado por baixa latência e alta confiabilidade, enquanto o eMBB é rotulado como classe 1 e caracterizado por altas taxas de dados, e ambos são gerados na VM 1 pelo UE, atravessando o núcleo 5GC na VM 2 com destino à rede de dados externa na VM 3. O modelo foi treinado utilizando esses rótulos e as seguintes características:

- Média do IAT do fluxo completo;
- Variância do IAT do fluxo completo;
- Média do IAT na janela dos últimos 500 ms;
- Variância do IAT na janela dos últimos 500 ms.

O trabalho não detalha os parâmetros específicos dos geradores de tráfego, informando somente que o tráfego eMBB é gerado por uma aplicação de *adaptive video streaming* hospedada na VM 1 e que o URLLC simula controle de um braço robótico também na VM 1.

As métricas de uso da CPU capturam o impacto do processamento no espaço do usuário, nas rotinas de *kernel* e nas interrupções de *software* de rede. A acurácia do modelo no ambiente *kernel* é calculada por métricas como precisão, *Recall* e *F1-score*, comparando os resultados obtidos *in-kernel* com uma implementação de referência no espaço do usuário.

Como resultado, o modelo foi capaz de realizar a classificação 36x mais rápido quando comparado à utilização do espaço do usuário, além de reduzir o consumo total de CPU de 15,79% para 12,87% e manter uma acurácia de 99,6%.

A. Aplicação no Ambiente Configurado

Para execução dos classificadores desenvolvidos em [2], disponíveis em um repositório GitHub do autor Jose S. S. Galvão [4], no ambiente virtualizado e dockerizado deste trabalho, foram necessárias adaptações devido a incompatibilidade de versão de *software*. Algumas das diferenças são listadas na Tabela V, onde a versão da biblioteca BPF usada, o *software* do núcleo 5G e o modo XDP utilizado foram fatores limitantes.

TABELA V
DIFERENÇAS ENTRE O AMBIENTE 5G DESTE TRABALHO E O DO TRABALHO [2]

Característica	**Este trabalho**	**Trabalho [2]**
Sistema operacional da(s) VM(s)	22.04 LTS	20.04
Software para o 5GC	Open5GS	Free5GC
Modo XDP utilizado	Genérico	Não especificado

O BCC é utilizado por ambos os classificadores, no espaço do *kernel* e no espaço do usuário, para compilar e carregar os programas BPF em tempo de execução. O sistema operacional da VM utilizada neste trabalho disponibiliza a versão 0.18.0 do BCC, que é incompatível com o *kernel* 5.15.0.176 da VM. Essa incompatibilidade causa erro de compilação do programa BPF do espaço do *kernel*.

Como solução, o BCC 0.36.1 foi compilado a partir do código fonte disponível no repositório oficial do projeto*iovisor* [5] e instalado sem sobrescrever a versão do sistema. E dessa forma a execução do classificador no espaço do *kernel* foi realizada com a especificação explicita das variáveis de ambiente necessárias, com o comando abaixo:

```
LD_LIBRARY_PATH=/opt/bcc/lib \
    PYTHONPATH=/opt/bcc/lib/python3/dist-
packages/bcc-0.36.1+a14e204d-py3.10.egg \
    python3 monitorjanela.py
```

1) Classificador no Kernel: O classificador no espaço do *kernel* não pôde ser executado dentro do contêiner da UPF. Embora tenhamos contornado o problema da versão do BCC na VM, a versão 0.36.1 do BCC não pôde ser carregada para o contêiner devido à incompatibilidade do GLIBC da imagem do Open5GS utilizada (Debian 11). A solução para esse impedimento seria utilizar outra imagem para a UPF ou compilar outra versão do GLIBC no contêiner, mas ambas as alternativas exigiriam reestruturação do ambiente. Diante disso, foi avaliado que, como o objetivo é verificar a acurácia do modelo, que independe de ser executado no contêiner ou no *host*, e o uso de CPU da abordagem em *kernel*, que deve ser medido a partir do *host*, como explicado na Subseção IV-C.3, o classificador do espaço do *kernel* pode ser executado no *host*.

2) Classificador no espaço do usuário: Como já mencionado, o classificador de espaço do *kernel* foi executado a partir do *host*, não do contêiner da UPF, por isso, a mesma conduta foi adotada para o classificador no espaço do usuário. E, para a sua execução, o binário *libclassifier.so* disponibilizado pelo autor foi descartado e recompilado localmente a partir do *wrapper.c* devido às incompatibilidades dos ambientes, com o seguinte comando:

```
gcc -shared -fPIC -o libclassifier.so wrapper.c
```

Além das alterações mencionadas, a interface utilizada nos códigos *monitor.py*, do espaço do usuário, e *monitorjanela.py*, do espaço do *kernel*, foram substituídas pela interface virtual externa, disponível no *host*, do contêiner da gNB. Foi selecionada a interface da gNB e não da UPF pois na *veth* externa da UPF flui o tráfego *downlink*, com origem na DN e destino no UE, enquanto na da gNB flui o *uplink*, que é o tráfego avaliado no trabalho em análise [2]. Isso foi confirmado monitorando a saída dos classificadores, que exibem os IPs de origem e de destino, em ambas as interfaces.

B. Metodologia para a Aplicação no Ambiente Configurado

Com o objetivo de avaliar a acurácia e o custo computacional dos classificadores, foram executados dois *scripts* separadamente, e ambos diferem somente quanto ao classificador executado. Os *scripts* executam dois geradores de tráfego, URLLC e eMBB, o classificador, no espaço do usuário ou

[4]https://github.com/jose-galvao/eBPF-Classification

[5]https://github.com/iovisor/bcc

do *kernel*, e a extração de métricas de consumo de CPU paralelamente, repetindo o processo 30 vezes e exportando arquivos com a saída dos programas de cada cenário e com a saída do `mpstat`.

Os geradores de tráfego são dois códigos em *Python* que utilizam exclusivamente bibliotecas nativas da linguagem (módulos `socket`, `time` e `argparse`), sendo um para gerar tráfego URLLC e outro para gerar tráfego eMBB. Os valores usados para a geração de tráfego foram baseados em [17], referência utilizada por [2] para definir o tráfego URLLC. O *script* para URLLC envia pacotes UDP periódicos com IAT constante de 8 ms e *payload* fixo de 128 bytes, produzindo um padrão de chegada regular e de baixa latência. Já o *script* do eMBB gera rajadas periódicas de 20 pacotes de 1000 bytes, com IAT de 0,5 ms entre pacotes e de 20 ms entre rajadas, produzindo um padrão de IAT bimodal e de alta variância, característica utilizada pelo modelo para identificação do tráfego eMBB.

Para calcular a acurácia dos modelos, os tráfegos foram gerados a partir de portas distintas. Como os programas imprimem as portas de origem do tráfego, diferir os dois tráfegos por elas torna possível identificar se o modelo errou ou acertou a classificação.

VI. Resultados e Discussão

Esta seção apresenta e discute os resultados obtidos a partir da metodologia especificada na seção IV e na subseção V-B.

A. Extração de Métricas de Rede com TC e XDP

A coleta de métricas de rede realizada pelos programas eBPF foi analisada e comparada a partir dos logs de saída na UPF. Foram removidos os registros durante os períodos de início do fluxo (*ramp-up*) e fim de fluxo (*ramp-down*) visando metrificar apenas as amostras em regime permanente. O processo removeu *outliers* que correspondiam a transientes.

TABELA VI
Comparação das métricas de rede entre XDP e TC (após remoção de outliers)

Métrica	XDP	TC
Amostras válidas	945 (87,5%)	917 (84,9%)
Throughput médio (Mbps)	104,000	103,998
Desvio padrão (Mbps)	0,022	0,024
Pacotes médio (pkt/s)	17856,95	17856,91
Desvio padrão (pkt/s)	3,68	4,08
Jitter médio (ms)	0,056	0,056

Como ilustrado na Tabela VI, a metrificação em diferentes pontos da pilha de rede (XDP no modo genérico e TC no ingresso) produziu resultados quase idênticos para as métricas de rede agregadas. Ambos os cenários apresentaram bom desempenho, sem perda de pacotes e com baixíssima variação. O XDP exibiu estabilidade ligeiramente superior (desvio padrão de 0,022 Mbps contra 0,024 Mbps do TC), porém a diferença é praticamente desprezível.

O resultado se encaixa no esperado, pois, ao operar no modo genérico, o XDP também é executado após a alocação do `sk_buff`, posicionando-se em patamar semelhante ao TC. Dessa forma, a diferença fundamental entre os dois ganchos passa a ser principalmente a flexibilidade de ações, como redirecionamento, e o custo de processamento no *kernel*. Para tarefas de inspeção e coleta de métricas, ambos os *hooks* entregam precisão equivalente, sendo a escolha determinada pelo consumo de CPU e pela compatibilidade com o *driver* de rede para o XDP, onde pode ocorrer *fallback* para o modo genérico caso não haja suporte [4].

B. Uso de CPU com TC e XDP

Após a validação das métricas de rede, procedeu-se para a análise da eficiência do sistema, visando mensurar o *overhead* computacional introduzido pela execução dos programas eBPF integrados à UPF. A investigação busca contrastar o impacto das funções processadas diretamente no contexto do *kernel* com as abordagens operando no espaço do usuário.

Foram realizados cálculos e geradas visualizações estatísticas baseadas na distribuição da carga de processamento entre as métricas %usr, %sys e %softirq a partir das saídas do `mpstat`, permitindo identificar como a programabilidade do plano de dados afeta o uso dos recursos da CPU.

No cenário de referência (*baseline*), os pacotes GTP-U foram processados por meio do caminho padrão da pilha de rede do *kernel* Linux, sem a execução de monitoramento. Os resultados apresentados na Tabela VII indicam baixos níveis de utilização da CPU, com predominância do estado ocioso (%idle = 91,94%). Os percentuais observados em espaço de usuário (%usr = 1,87%), espaço de sistema (%sys = 3,75%) e interrupções de *software* (%soft = 2,44%) estão associados às operações nativas da pilha de rede, ao processamento do tráfego pela UPF e à infraestrutura do ambiente baseada em contêineres Docker.

TABELA VII
Uso médio de CPU por cenário

Cenário	%usr	%sys	%soft	CPU Total
Baseline	1,87	3,75	2,44	8,06
XDP	3,32	6,69	4,50	14,51
TC	3,26	6,53	4,20	13,99
tcpdump	4,32	8,51	4,10	16,93

TABELA VIII
Uso médio de CPU por cenário (medianas)

Cenário	%usr	%sys	%soft	CPU Total
Baseline	1,10	2,13	1,15	4,38
XDP	3,25	6,42	4,11	13,78
TC	3,34	6,87	4,20	14,41
tcpdump	4,01	7,88	3,44	15,33

As análises por média de uso de CPU comprovam o maior *overhead* do `tcpdump` executado no espaço do usuário. Em uma visualização mais detalhada, como ilustrado nas Fig. 8 a 10, é possível verificar a estabilidade das amostras em diferentes

camadas de execução sob um nível de confiança de 95%[6]. Os gráficos revelam a eficiência dos programas executados no espaço do *kernel* para extração de métricas, posicionando-se abaixo do `tcpdump` em termos de *overhead*.

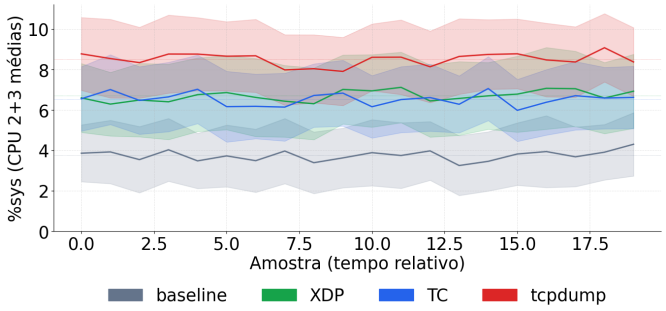


Fig. 8
VISUALIZAÇÃO COMPARATIVA DO USO DE CPU NO ESPAÇO DO *kernel* COM INTERVALO DE CONFIANÇA DE 95% (%SYS)

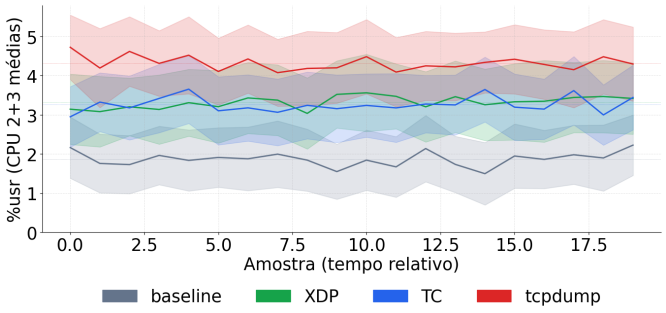


Fig. 9
VISUALIZAÇÃO COMPARATIVA DO USO DE CPU NO ESPAÇO DO USUÁRIO COM INTERVALO DE CONFIANÇA DE 95% (%USR)

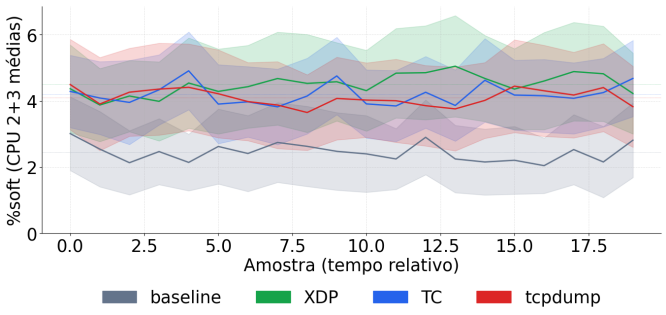


Fig. 10
VISUALIZAÇÃO COMPARATIVA DO USO DE CPU NO ESPAÇO DO *kernel* COM INTERVALO DE CONFIANÇA DE 95% (%SOFT)

Embora os cenários baseados em eBPF apresentem valores ligeiramente superiores de %soft em relação ao `tcpdump`, esse comportamento é compatível com a arquitetura da pilha de rede Linux. O %soft representa o tempo de CPU consumido pelo processamento de interrupções de *software* (*softirqs*). Como os programas em XDP e TC são executados diretamente no caminho de processamento dos pacotes, após a alocação do `sk_buff`, cada pacote aciona instruções adicionais para coleta e contabilização de métricas, aumentando o tempo associado às *softirqs*. O `tcpdump`, a partir da biblioteca *libpcap*, transfere parte do processamento para o espaço do usuário, o que resulta em maiores valores de %usr e %sys.

Outra característica notável observada a partir das saídas do `mpstat` é a similaridade de desempenho entre os cenários TC e XDP. A partir da Tabela IX, observa-se que os intervalos de confiança dos dois apresentam ampla sobreposição e medianas muito próximas, indicando diferenças pouco expressivas no consumo de CPU. Esse comportamento é consistente com o ambiente experimental, uma vez que o XDP foi executado em modo genérico.

TABELA IX
INTERVALOS DE CONFIANÇA (95%) DA UTILIZAÇÃO DE CPU POR CENÁRIO

Cenário	**%usr**	**%sys**	**%soft**
Baseline	[1,71 ; 2,04]	[3,42 ; 4,07]	[2,22 ; 2,67]
XDP	[3,13 ; 3,51]	[6,31 ; 7,07]	[4,22 ; 4,78]
TC	[3,09 ; 3,44]	[6,18 ; 6,87]	[3,96 ; 4,44]
tcpdump	[4,13 ; 4,51]	[8,13 ; 8,88]	[3,83 ; 4,37]

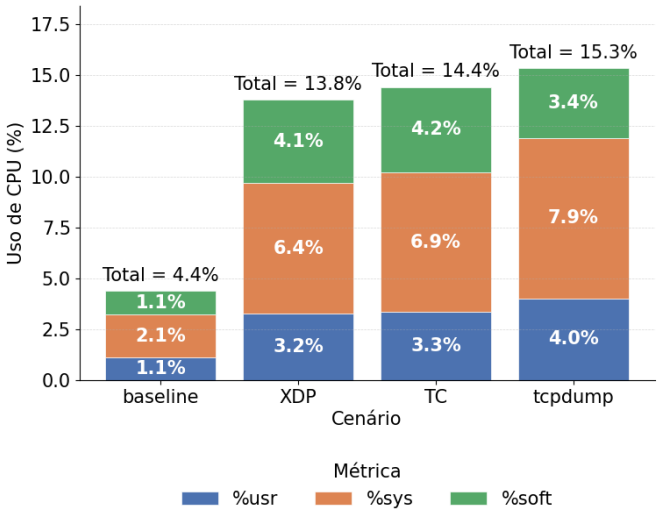


Fig. 11
VISUALIZAÇÃO EM BARRAS DO USO DE CPU COM DISTRIBUIÇÃO DA CARGA DE PROCESSAMENTO ENTRE AS CAMADAS DE EXECUÇÃO BASEADO NA MEDIANA

C. Resultados da Reprodução do Caso de Uso

Embora os classificadores em ambos os cenários realizem a classificação a partir da avaliação da média e da variação do IAT do fluxo completo e dos últimos 500 ms, conforme exposto em [2], eles se diferem, além do espaço de execução,

[6]Os intervalos de confiança foram calculados a partir da distribuição t de Student, utilizando o erro padrão da média das 30 execuções realizadas para cada cenário.

em relação ao objeto alvo da classificação: a partir dos códigos disponibilizados foi observado que o cenário do espaço do *kernel* realiza a classificação por pacote, enquanto o cenário do espaço do usuário realiza a classificação de um conjunto de pacotes a cada 500 ms. Portanto, a comparação de acurácia do modelo executado no *kernel* e no espaço do usuário não é completamente justa.

O classificador no espaço do *kernel* processou em média 62801 pacotes por iteração, com uma média de 7766 pacotes de tráfego URLLC e 55035 pacotes de tráfego eMBB. O classificador no espaço do usuário processou em média 199,6 janelas de 500 ms por iteração, sendo aproximadamente 99,6 janelas de tráfego URLLC e 99,9 janelas de tráfego eMBB. A diferença no volume de unidades processadas é consequência direta da granularidade mencionada: o classificador no *kernel* opera por pacote, enquanto o classificador no espaço do usuário opera por janela temporal.

Em relação à acurácia, os classificadores em ambos os cenários tiveram altas taxas de acerto ao longo das 30 iterações. A classificação no espaço do *kernel* teve uma acurácia média de 99,98%, enquanto no espaço do usuário teve uma acurácia de 100%, uma diferença de 0,02%. Os resultados do cenário do *kernel* mostram que a acurácia para o tráfego URLLC foi de 100% em todas as iterações, enquanto para o tráfego eMBB a média e a mediana foram de 99,973%, com variação entre 99,957% e 99,99%, como mostra a Fig. 12. Vale ressaltar que para a reproduçao do experimento foi utilizado tráfego controlado, sem uso de aplicações reais, portanto, a variação do IAT é artificial e definida antes da execução. O resultado obtido no cenário do *kernel* evidencia o potencial do uso do *hook* XDP do eBPF para aplicações de rede, pois mesmo com a limitação de não poder usar aritmética de ponto flutuante ele apresentou acurácia próxima de 100%.

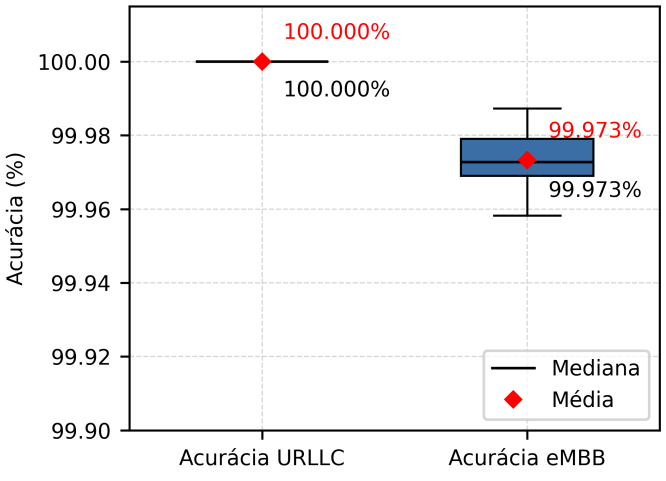


Fig. 12
COMPOSIÇÃO DA ACURÁCIA DO MODELO NO ESPAÇO DO KERNEL

O classificador no espaço do usuário registrou 100% de acurácia para os dois tipos de tráfego em todas as 30 iterações. Esse resultado é consistente com o esperado, pois o classificador no espaço do usuário, além de operar por janela temporal e consequentemente ter realizado menos classificações, ele utiliza aritmética de ponto flutuante, sem as restrições impostas pelo verificador eBPF, o que preserva a precisão do modelo.

A classificação no espaço do *kernel* utilizando a abordagem desenvolvida em [2] se mostrou promissora para cenários que exigem baixa latência, como serviços URLLC, pois, além da ótima acurácia, ele apresentou baixíssimo tempo de inferência, com média de 0,22 μs, como evidencia a Fig. 13. Além disso, vale destacar que o modelo *in-kernel* replicado neste trabalho utilizou o XDP no modo genérico, e que se utilizado no modo nativo haveria vantagem no que diz respeito ao uso de CPU. E, como já mencionado, o classificador no espaço do usuário atua por janela temporal, fator que influencia o tempo de inferência do modelo, que registrou média de 59,41 μs.

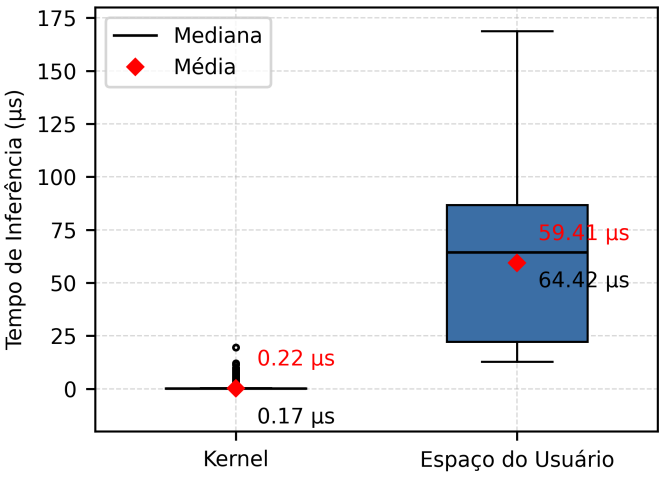


Fig. 13
TEMPO DE INFERÊNCIA DOS CLASSIFICADORES

O uso de CPU foi avaliado a partir das métricas `%usr`, `%sys` e `%soft` coletadas via `mpstat` ao longo das 30 iterações de cada cenário, considerando o conjunto total de CPUs do sistema. O uso total foi calculado como a soma dessas três métricas, de forma consistente com a metodologia adotada em [2], mas diferindo quanto ao escopo utilizado para a extração das métricas. O trabalho em análise realizou a medição de uso de CPU em uma VM que hospedava a UPF e o núcleo 5G, enquanto neste trabalho a extração foi feita em uma VM que hospeda todos os componentes da rede, portanto, os valores obtidos incluem o custo computacional de todos os processos da VM, incluindo, por exemplo, o custo da geração de tráfego no UE. Além disso, todos os valores de métrica obtidos se sobrepõem entre os cenários, portanto não é possível afirmar conclusivamente superioridade entre os cenários.

O cenário de *baseline*, operando apenas com os geradores de tráfego e sem classificador ativo, apresentou uso total médio de 8,32%, servindo como referência para os cenários de operação do classificador.

O classificador no espaço do *kernel* apresentou uso total médio de 8,76%. O `%sys` e o `%soft` médios foram de 1,11% e 0,16%, respectivamente, próximos ao registrado no *baseline*, como mostra a Tabela X. Ressalta-se que, diferente do trabalho de [2], o programa eBPF foi anexado ao *hook* XDP no modo genérico, no qual os pacotes já foram processados pelo *driver*

e pelo subsistema de rede antes de serem interceptados pelo *hook*. Por esse motivo, não se espera redução significativa de interrupções de *software* em relação ao *baseline*, ao contrário do que se esperaria com o XDP nativo.

TABELA X
USO MÉDIO DE CPU

Cenário	*Baseline*	Espaço do *kernel*	Espaço do usuário
`%usr`	7,05% ± 2,11%	7,49% ± 2,13%	7,32% ± 2,06%
`%sys`	1,06% ± 1,05%	1,11% ± 1,41%	1,59% ± 1,44%
`%soft`	0,20% ± 0,29%	0,16% ± 0,28%	0,30% ± 0,46%
Total	**8,32% ± 2,61%**	**8,76% ± 2,93%**	**9,20% ± 3,12%**

Vale ressaltar também o motivo de o `%soft` ter se mantido estável na reprodução do caso de uso, enquanto nos resultados do estudo comparativo, apresentados na seção VI-B, houve um aumento de 2,44% do *baseline* para 4,50% no cenário com XDP. Essa diferença reflete o volume de pacotes gerados em cada experimento, que, para a reprodução do classificador *in-kernel* foram gerados em média 2093,38 pacotes por segundo, contra 17856,95 pacotes por segundo no estudo comparativo. Com esse menor volume de tráfego e a extração das métricas considerando todos os processos da VM, o *overhead* do XDP se tornou imperceptível.

O classificador no espaço do usuário apresentou uso total médio de 9,20%, contra 8,76% no *kernel*. O `%soft` foi o mais elevado entre os três cenários, condizente com os resultados apresentados em [2], diante da necessidade de troca de dados entre o espaço do *kernel* e o espaço do usuário.

O `%usr` apresentou valores similares nos três cenários,*Baseline* 7,05%, *kernel* 7,49% e Espaço do Usuário 7,32%, sem diferença expressiva. Esse comportamento é atribuído à limitação do ambiente experimental: o `%usr` é dominado pelo custo dos geradores de tráfego e pelas funções de rede de controle do Open5GS, diluindo as diferenças que seriam observadas em um ambiente com máquinas dedicadas como o de [2].

As Fig. 15 e 14 evidenciam a alta variabilidade do ambiente devido a característica de *host* compartilhado, onde processos externos como as NFs de controle introduzem ruído periódico. Ainda assim, a tendência geral de o cenário no espaço do usuário apresentar maior uso de CPU é visível ao longo das 30 iterações, sendo consistente com o *overhead* de trocas de contexto inerente à arquitetura de classificação no espaço do usuário.

VII. CONCLUSÕES

Este trabalho apresentou uma análise do uso do eBPF como mecanismo de monitoramento e observabilidade em redes 5G baseadas em *software*, especificamente na UPF, conduzida sobre um ambiente experimental virtualizado e conteinerizado com UERANSIM e Open5GS. A avaliação contemplou tanto o uso passivo do eBPF, medindo a sobrecarga do sistema com a inspeção de pacotes nos *hooks* XDP e TC comparado ao `tcpdump`, quanto um caso de uso ativo, com a classificação de tráfego em tempo real por meio de um modelo de aprendizado de máquina.

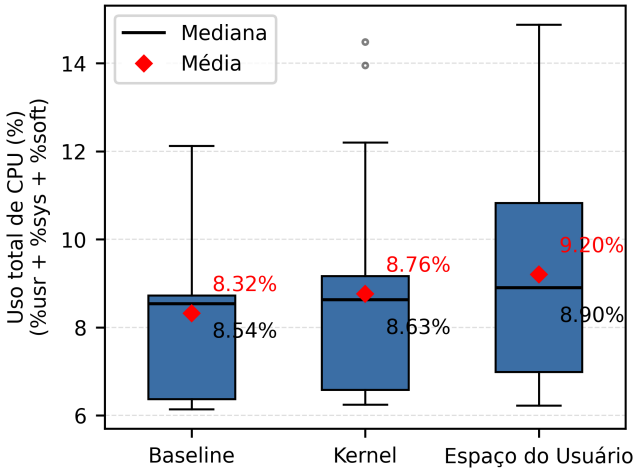


Fig. 14
USO DE CPU NOS ESPAÇOS DO *Kernel* E DO USUÁRIO

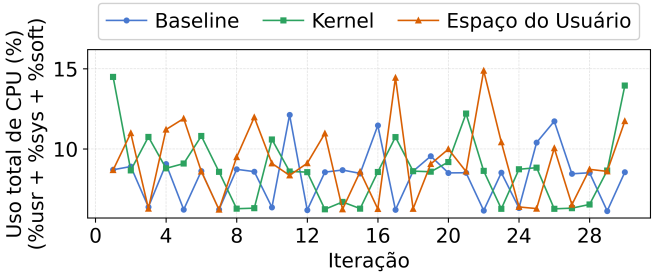


Fig. 15
VARIAÇÃO DO USO DE CPU POR ITERAÇÃO

Os resultados demonstraram que as abordagens baseadas em eBPF foram capazes de realizar a coleta de métricas de *throughput* e *Jitter* com precisão equivalente, apresentando valores praticamente idênticos para as métricas de rede observadas. Em termos de eficiência computacional, os cenários utilizando eBPF apresentaram menor consumo de CPU quando comparados ao `tcpdump`, evidenciando as vantagens da execução de funções de monitoramento diretamente no contexto do *kernel*. A análise dos gráficos também mostrou que os resultados obtidos pelos *hooks* XDP e TC foram muito próximos entre si, comportamento compatível com a utilização do XDP em modo genérico, no qual parte das vantagens do processamento antecipado é reduzida.

A implementação do caso de uso demonstrou também a viabilidade prática da utilização do eBPF na UPF para classificação de tráfego diretamente no plano de dados. Apesar de o ambiente experimental não reproduzir integralmente as condições do trabalho original, devido a restrições de *software* e diferenças na infraestrutura, o sistema foi capaz de executar o processo de classificação e permitiu a observação das vantagens associadas à abordagem *in-kernel*.

Como trabalhos futuros, pretende-se avaliar os programas utilizando XDP em modo nativo bem como quantificar sua

diferença ao modo genérico. Além disso, visa-se expandir a aplicação do eBPF para atuação na rede, como para balanceamento de carga e failover de UPF.

AGRADECIMENTOS

Os autores agradecem à Escola de Engenharia Elétrica, Mecânica e de Computação (EMC) da Universidade Federal de Goiás (UFG), especialmente ao Professor Orientador Flávio Geraldo Coelho Rocha e a Jonas Augusto Kunzler, pelo apoio à realização deste trabalho.

REFERÊNCIAS

[1] S. N T, D. Dalai, and B. Manoj, "Towards 6g: A 5g system latency analysis," in *2025 IEEE Future Networks World Forum (FNWF)*. Bangalore, India: IEEE, 2025.

[2] J. S. S. Galvão, F. G. C. Rocha, R. D. Gomes, and L. C. Almeida, "Leveraging ebpf/xdp for real-time machine learning traffic classification in 5g user plane networks," in *Anais do 1º Workshop de Inteligência Artificial para Redes de Computadores (WIARC)*. Porto Alegre: Sociedade Brasileira de Computação, 2026, pp. 99–112. [Online]. Available: https://doi.org/10.5753/wiarc.2026.23145

[3] 3rd Generation Partnership Project, "5g system overview," 2022, Disponível em: <https://www.3gpp.org/technologies/5g-system-overview>. Acesso em: 29 de maio de 2026.

[4] eBPF Community, "ebpf documentation," acessado em: 28 de maio de 2026. [Online]. Available: https://ebpf.io/what-is-ebpf

[5] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, and D. Ahern, "The express data path: fast programmable packet processing in the operating system kernel," in *Proceedings of the 14th International Conference on emerging Networking EXperiments and Technologies*, ser. CoNEXT '18. New York, NY, USA: ACM, 2018, pp. 54–66.

[6] E. Dahlman, S. Parkvall, and J. Sköld, *5G NR: The Next Generation Wireless Access Technology*, 2nd ed. Academic Press, 2021.

[7] ETSI, "5G; System architecture for the 5G System (5GS) (3GPP TS 23.501 version 18.7.0 Release 18)," European Telecommunications Standards Institute (ETSI), Sophia Antipolis, France, Technical Specification (TS) ETSI TS 123 501 V18.7.0, 10 2024.

[8] M. A. M. Vieira, M. S. Castanho, R. D. G. Pacífico, E. R. S. Santos, E. P. M. Câmara Júnior, and L. F. M. Vieira, "Fast packet processing with ebpf and xdp: Concepts, code, challenges and applications," *ACM Computing Surveys (CSUR)*, 2019, universidade Federal de Minas Gerais (UFMG).

[9] D. Soldani, P. Nahi, H. Bour, S. Jafarizadeh, M. F. Soliman, L. Di Giovanna, F. Monaco, G. Ognibene, and F. Risso, "ebpf: A new approach to cloud-native observability, networking and security for current (5g) and future mobile networks (6g and beyond)," *IEEE Access*, vol. 11, pp. 54 943–54 972, 2023.

[10] L. Rice, *Learning eBPF: Programming the Linux Kernel for Enhanced Observability, Networking, and Security*, 1st ed. O'Reilly Media, 2023.

[11] W. Choi, Y. Yoo, K. Lee, Z. Niu, P. Cheng, Y. Xiong, G. Yang, and C. Yoo, "Intelligent packet processing for performant containers in iot," *IEEE Internet Of Things*, vol. 11, no. 24, pp. 40 388–40 403, 2024.

[12] Z. Suo, Y. Zhao, W. Chen, and J. Rao, "An analysis and empirical study of container networks," in *IEEE INFOCOM 2018 - IEEE Conference on Computer Communications*. Honolulu, HI, USA: IEEE, April 2018, pp. 189–197.

[13] T. Ozturk, "Performance evaluation of express data path for container-based network functions," Master's thesis, Aalto University, School of Electrical Engineering, Espoo, Finland, 2021. [Online]. Available: http://urn.fi/URN:NBN:fi:aalto-202101311791

[14] C. Scheich, M. Corici, H. Buhr, and T. Magedanz, "Performance analysis of a 5g user plane function accelerated with express data path in docker containers," in *2023 IEEE Future Networks World Forum (FNWF)*, 2023, pp. 1–6.

[15] eBPF Community, "ebpf libraries - libbpf," acessado em: 28 de maio de 2026. [Online]. Available: https://docs.ebpf.io/ebpf-library/libbpf/

[16] The Kernel Development Community, "Scaling in the linux networking stack," 2021, linux 5.15 Documentation. [Online]. Available: https://www.kernel.org/doc/html/v5.15/networking/scaling.html

[17] A. E. Kalør, R. Guillaume, J. J. Nielsen, A. Mueller, and P. Popovski, "Network slicing in industry 4.0 applications: Abstraction methods and end-to-end analysis," *IEEE Transactions on Industrial Informatics*, vol. 14, no. 12, pp. 5419–5427, 2018.